Doc. no. P0004R1
Date: 2015-10-19
Project: Programming Language C++
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Remove Deprecated iostreams aliases

Revision History

Revision 0

Original version of the paper for the 2015 pre-Kona mailing.

Revision 1

Added a compatibility clause for LWG review.

I. Table of Contents

- Revision History
  - Revision 0
  - Revision 1
- I. Table of Contents
- II. Introduction
- III. Motivation and Scope
- IV. Compatibility
- V. Impact On the Standard
- VII. Technical Specification
- VII. References

II. Introduction

Annex D.6 Old iostreams members [depr.ios.members] contains a number of aliases for types and functions in the iostreams library facility for compatibility with similar pre-standard libraries. They were deprecated in the original 1998 C++ standard, and it is time to formally remove them.

III. Motivation and Scope

As the C++ standard grows larger, there is an incremental cost in maintaining pages of obsolete features. This was acknowledged at the Urbana meeting with the removal of several deprecated library clauses, such `auto_ptr` and the classic function binders. The the list of references at the end of this document for a list of other deprecated features expected to be removed from the next version of the standard.

This paper proposes the removal of some names that are purely aliases of existing library names, that many readers of the standard are not aware of due to their presence only in the deprecated features annex, and not actually present in their corresponding class definitions in clause 27.

This paper specifically does *not* recommend removing clause D.7 though, the `strstream` library. That proposal has been made several times before, and rejected each time for lack of a replacement feature to solve the same problems as efficiently.

IV. Compatibilty

There will likely be an impact on legacy user code that has not been updated to modernize use of these names in the ~20 years they will have been deprecated when the next standard is released. However, it is expected to be a much lower impact than the already approved removal of `auto_ptr` and the classic function binders.

None of the functions are virtual, so there should be no ABI compatibilty issues. As with any library change, there is always a risk that the specific identifiers presence or absence could be detected with SFINAE trickery (such as the recently adopted `void_t`) which may be observable through an ABI, but such concerns would inhibit all changes to the standard library, and would be

considered an even lower risk than usual in this case, as the names are intended only for compatibility with code that predates the discovery of SFINAE-based reflection.

V. Impact On the Standard

The impact on the standard is minimal, as no other part of the standard assumes these names exist.

VI. Technical Specifications

Strike the entirety of Annex D.6 Old iostream members [depr.ios.members].

~~D.6 Old iostreams members [depr.ios.members]~~

~~1 The following member names are in addition to names specified in Clause 27:~~

```
namespace std {
  class ios_base {
  public:
    typedef T1 io_state;
    typedef T2 open_mode;
    typedef T3 seek_dir;
    typedef implementation-defined streamoff;
    typedef implementation-defined streampos;
    // remainder unchanged
  };
}
```

~~2 The type `io_state` is a synonym for an integer type (indicated here as *T1*) that permits certain member functions to overload others on parameters of type `iostate` and provide the same behavior.~~

~~3 The type `open_mode` is a synonym for an integer type (indicated here as *T2*) that permits certain member functions to overload others on parameters of type `openmode` and provide the same behavior.~~

~~4 The type `seek_dir` is a synonym for an integer type (indicated here as *T3*) that permits certain member functions to overload others on parameters of type `seekdir` and provide the same behavior.~~

~~5 The type `streamoff` is an implementation-defined type that satisfies the requirements of off_type in 27.2.2.~~

~~6 The type `streampos` is an implementation-defined type that satisfies the requirements of pos_type in 27.2.2.~~

~~7 An implementation may provide the following additional member function, which has the effect of calling `sbumpc()` (27.6.3.2.3):~~

```
namespace std {
  template<class charT, class traits = char_traits<charT> >
  class basic_streambuf {
  public:
    void stossc();
    // remainder unchanged
  };
}
```

~~8 An implementation may provide the following member functions that overload signatures specified in Clause 27:~~

```
namespace std {
  template<class charT, class traits> class basic_ios {
  public:
    void clear(io_state state); void setstate(io_state state); void exceptions(io_state);
    // remainder unchanged
  };

  class ios_base {
  public:
    // remainder unchanged
  };

  template<class charT, class traits = char_traits<charT> >
  class basic_streambuf {
  public:
    pos_type pubseekoff(off_type off, ios_base::seek_dir way,
                        ios_base::open_mode which = ios_base::in | ios_base::out);
    pos_type pubseekpos(pos_type sp,
```

```
            ios_base::open_mode which);
    // remainder unchanged
};

template <class charT, class traits = char_traits<charT> >
class basic_filebuf : public basic_streambuf<charT,traits> {
public:
    basic_filebuf<charT,traits>* open
    (const char* s, ios_base::open_mode mode);
    // remainder unchanged
};

template <class charT, class traits = char_traits<charT> >
class basic_ifstream : public basic_istream<charT,traits> {
public:
    void open(const char* s, ios_base::open_mode mode);
    // remainder unchanged
};

template <class charT, class traits = char_traits<charT> >
class basic_ofstream : public basic_ostream<charT,traits> {
public:
    void open(const char* s, ios_base::open_mode mode);
    // remainder unchanged
};
}
```

9 The effects of these functions is to call the corresponding member function specified in Clause 27.

Insert the following compatibility clause into Annex C:

C.4.2 Annex D: compatibility features [diff.cpp14.depr]

D.6

Change: Remove old iostreams members [depr.ios.members].

Rationale: Redundant feature for compatibility with pre-standard code has served its time.

Effect on original feature: A valid C++ 2014 program using these identifiers may be ill-formed in this International Standard.

VII. References

- N4086 Removing trigraphs??!, Richard Smith
- N4168 Removing `auto_ptr`, Billy Baker
- N4190 Removing `auto_ptr`, `random_shuffle()`, And Old `<functional>` Stuff, Stephan T. Lavavej
- P0001R0 Remove Deprecated Use of the `register` Keyword
- P0002R0 Remove Deprecated `operator++(bool)`